Demand Forecasting for Phone Devices Report

Completed by: Vu Duc Cong

1. Introduction

The goal of this project is to forecast the demand for phone devices, measured as Closing Subs Monthly, across different countries, models, and configurations. This involves training machine learning models to predict the next two months of demand using historical features.

2. Data Understanding

- Data imported from Excel.
- Initial column exploration performed
- Assumptions made regarding column meanings and relationships.
- A dictionary was manually constructed for reference.

Column x Column Explaination

[0] YearMonth: The time period in YYYYMM format (e.g., 201901 = Jan 2019). Use this as your time index.

[1] country :Geographic market (e.g., "California" – represent a state).

[2] Make: The brand of the phone (e.g., Oppo, Samsung, Apple).

[3] Phone Series: The specific model variant (e.g., OPPO A10). Could be used to group models.

[4] PHONE_LAUNCH_DATE: Date the phone model was released to market. Helps calculate age, lifecycle stage, etc.

[5] ModelFamily: The group or family to which this phone belongs.

[6] Predecessor: The previous model in the series (may be NaN for first model).

[7] Successor: The next model in the series (may help understand demand drop-off or cannibalization).

[8] Model Age (Days): Number of days since launch (i.e., current date - launch date).

[9] Model Age (Months): Number of months since launch. Useful for modeling lifecycle impact on demand.

[10] Model: Full model name including memory and color (e.g., A12 32GB BLUE)

[11] SKU: Stock Keeping Unit – unique ID for this specific variant (e.g., A12BLU)

[12] SKU No Colour: Model ID excluding color — e.g., all A12 variants

[13] Color: Color of the phone

[14] Size: Storage capacity (e.g 32GB)

[15] Closing Subs Monthly: Likely be the number of units sold closed in that month. This could be the target variable for forecasting demand.

[16] Filed Claims: Number of total warranty/support NEW claims filed that month for a given model

[17] Claims: Number of claims handled/resolved/handled that month

[18] Claims Swap: Number of claims resolved by providing same model (like-for-like)

[19] Claims Replacement: Number of claims resolved by providing a different model

[20] IR Rate Swap: % of devices swapped per month (Claims Swap / Closing Subs × 100)

[21] IR Rate Replacement: % of devices replaced per month (Claims Replacement / Closing Subs x 100)

[22] IR Rate Monthly: Combined incident rate across all claim types for the model/month

[23] Churn: The number of users who stopped using the phone model in that month

[24] Churn Rate: The percentage of churned users relative to total users that month

3. Data Cleaning and Preprocessing

- Negative values found in “Model Age (Days)” were assumed to represent pre-orders

```
In [80]: # Percentage of the number of decive sold during pre-order (Order made before the Launch Date - Model Age (Days) < 0 )
         round(df[df['Model Age (Days)'] < 0]['Closing Subs Monthly'].sum()/df[df['Model Age (Days)'] >= 0]['Closing Subs Monthly'].sum()

Out[80]: 4.165
```

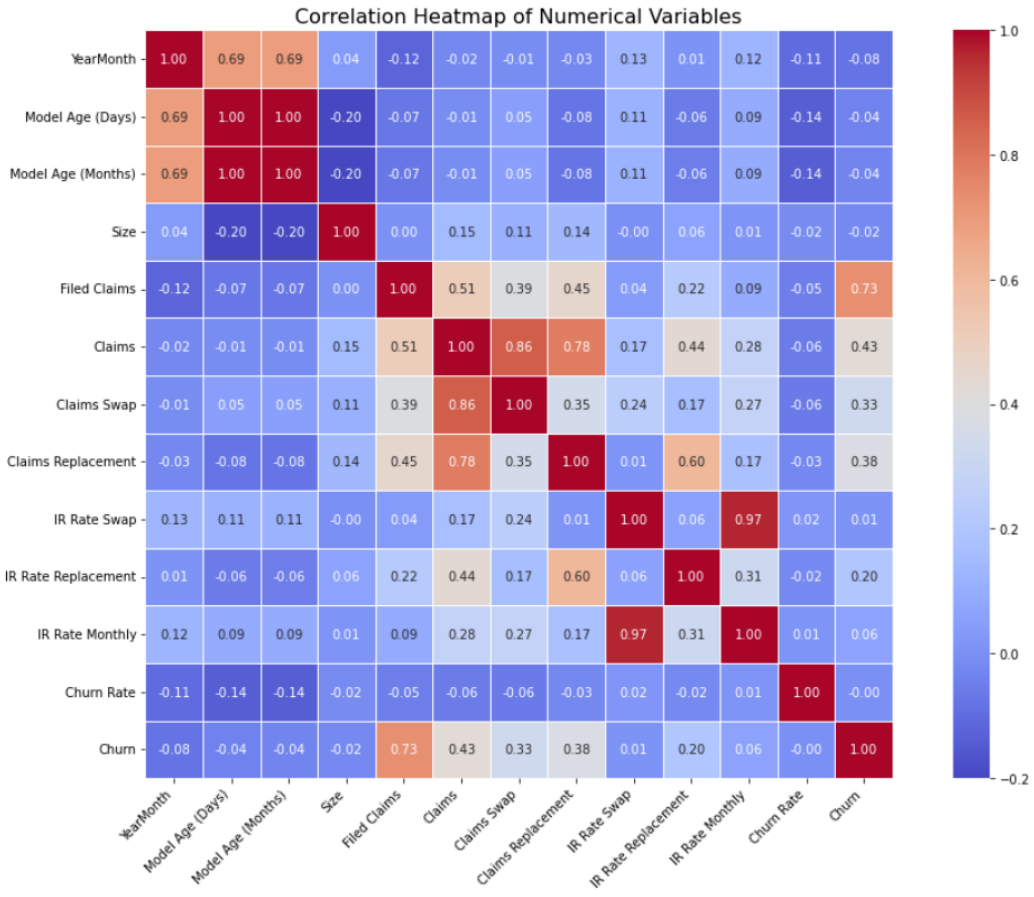
- Missing values replaced with zeros where appropriate
- Key ratio columns (IR Rate, Churn Rate) were calculated to ensure accuracy
- A global maximum YearMonth = 202311 was selected for modelling consistency

```
In [85]: df[df['YearMonth'] >= 202311].groupby(['country'])['Closing Subs Monthly'].mean()

Out[85]: country
         California    1125.829268
         Nevada        1740.212500
         Texas          736.688312
         Name: Closing Subs Monthly, dtype: float64

In [86]: df.groupby(['country','ModelFamily'])['YearMonth'].max().unique()

Out[86]: array([202311, 202310, 202305, 202308, 202309, 202304, 202210, 202211,
                202307], dtype=int64)

In [87]: # Step 1: Find the max YearMonth per (country, ModelFamily)
         max_months = df.groupby(['country', 'ModelFamily'])['YearMonth'].max().reset_index()

         # Step 2: Merge back to original dataframe to keep only rows where YearMonth is the max for that group
         df_max_months = df.merge(max_months, on=['country', 'ModelFamily', 'YearMonth'])

         # Step 3: Count how many records there are per each of those max YearMonths
         counts = df_max_months['YearMonth'].value_counts().sort_index()
         print(counts)

         202210      2
         202211      1
         202304      1
         202305      1
         202307      1
         202308      1
         202309      1
         202310      3
         202311    239
         Name: YearMonth, dtype: int64
```

- Records beyond this period were dropped to avoid prediction bias across countries

4. Feature Engineering

- A correlation heatmap was used to identify redundant numerical variables



- Numerical variables selected: Model Age (Days), Filed Claims, Claims Swap, Claims Replacement, Churn, Size
- Categorical variable selected: Country, ModelFamily, Colour

In [92]:
```
# After analysis, these are the following chosen columns for model training
# Feature Selection
numerical_selected_columns = [
    'YearMonth',
    'Model Age (Days)',
    'Filed Claims',
    'Claims Swap',
    'Claims Replacement',
    'Churn',
    'Size',
]

categorical_select_columns = [
    'country', 'ModelFamily', 'Colour'
]
```

- One-hot encoding applied using pd.get_dummies() & drop_first = True
- Time Index derived from Year and Month to maintain proper time series order
- Dataset sorted using TimeIndex and reindexed

5. Transformation & Splitting

- Numerical variables scaled using StandardScaler
- Data split temporally with shuffle = False

8. Data Splitting

In [100]:
```
# Split data for Training/ Validation & Testing
from sklearn.model_selection import train_test_split

# First: Train+Val and Test (70% / 30%)
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.30, shuffle=False  # keep temporal order
)

# Then: Train and Validation (from the 70%)
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.105, shuffle=False  # 60/15/25 split
)

print(f"Train: {len(X_train)}, Validation: {len(X_val)}, Test: {len(X_test)}")
```

```
Train: 8158, Validation: 958, Test: 3908
```

i. 60% training
ii. 15% validation
iii. 25% testing

6. Model Training and Evaluation

- Three models trained and validated:
    i. Linear Regression
    ii. Random Forest Regressor
    iii. XGBoost Regressor

In [102]:
```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

# Initialize models
models = {
    'Linear Regression': LinearRegression(),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
    'XGBoost': XGBRegressor(n_estimators=100, random_state=42)
}

# Train each model
for name, model in models.items():
    model.fit(X_train, y_train)
```

In [103]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Evaluation function
def evaluate(model, X, y):
    preds = model.predict(X)
    return {
        'MAE': mean_absolute_error(y, preds),
        'RMSE': mean_squared_error(y, preds, squared=False),
        'R2': r2_score(y, preds)
    }

# Validate each model
results = {}
for name, model in models.items():
    results[name] = evaluate(model, X_val, y_val)

# Show results
import pandas as pd
results_df = pd.DataFrame(results).T
print("Validation Results:")
print(results_df.round(3))
```

- Evaluation metrics:
    i. MAE (Mean Absolute Error)

ii. RMSE (Root Mean Squared Error)
iii. R^2 (Coefficient of Determination)

- Initial Performance:

Key Evaluation Metric			
	MAE	RMSE	R^2
Linear Regression	612.719	819.332	0.805
Random Forest	276.286	506.758	0.925
XGBoost	285.978	484.270	0.932

7. Forecasting Next Two Months (2023-12, 2024-01)

- Based on rows in 202311, cloned inputs for the next 2 months
- YearMonth advanced by +31 days for 202312 and 62 days for 202401
- Manually entered projected values for numerical variables

In [106]:
```
latest_data = df[df['YearMonth'] == 202311].copy()
categorical_selected_features = [
    'country', 'ModelFamily', 'Colour'
]
numerical_selected_features = ['Model Age (Days)',
 'Filed Claims',
 'Claims Swap',
 'Claims Replacement',
 'Churn',
 'Size']
```

In [107]:
```
# Filter only the latest data available (202311)
# Keep only the needed columns
cat_vars = categorical_select_columns + ['YearMonth']
future_base = latest_data[cat_vars].copy()

# Create future month rows
future_data = pd.concat([future_base]*2, ignore_index=True)

# Assign future months
future_data['YearMonth'] = [202312]*len(future_base) + [202401]*len(future_base)

future_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 478 entries, 0 to 477
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   country      478 non-null    object
 1   ModelFamily  478 non-null    object
 2   Colour       478 non-null    object
 3   YearMonth    478 non-null    int64 
dtypes: int64(1), object(3)
memory usage: 15.1+ KB
```

In [108]:
```
# Manually define values (with robust conditional logic)
# Start by repeating from 202311 base
future_data['Model Age (Days)'] = latest_data['Model Age (Days)'].tolist() * 2
future_data['Size'] = latest_data['Size'].tolist() * 2  # Keep original sizes

# Conditionally add model age by future month
future_data.loc[future_data['YearMonth'] == 202312, 'Model Age (Days)'] += 31
future_data.loc[future_data['YearMonth'] == 202401, 'Model Age (Days)'] += 62

# Manually set other features
future_data['Filed Claims'] = 2
future_data['Claims Swap'] = 0
future_data['Claims Replacement'] = 0
future_data['Churn'] = 0
```

- Used trained Random Forest and XGBoost models to generate forecasts

In [113]:
```
# Make predictions using XG Boost
future_predictions = models['XGBoost'].predict(future_encoded)

# Output
forecast_result_1 = future_data[['YearMonth', 'country', 'ModelFamily']].copy()
forecast_result_1['Predicted Closing Subs Monthly'] = future_predictions

print(forecast_result_1.head(10))
```

```
   YearMonth     country ModelFamily  Predicted Closing Subs Monthly
0     202312  California    OPPO A12                     -43.704189
1     202312       Texas    OPPO A12                     129.454697
2     202312  California    OPPO A15                      68.050888
3     202312  California    OPPO A15                     -71.584686
4     202312  California    OPPO A16                     319.590729
5     202312  California    OPPO A16                     342.318634
6     202312       Texas    OPPO A16                     362.251984
7     202312       Texas    OPPO A16                     362.251984
8     202312       Texas    OPPO A16                     353.466492
9     202312       Texas    OPPO A16                     353.466492
```

In [116]:
```
# Make predictions using Random Forest
rf_predictions = models['Random Forest'].predict(future_encoded)

# Output
forecast_result_2 = future_data[['YearMonth', 'country', 'ModelFamily']].copy()
forecast_result_2['Predicted Closing Subs Monthly'] = rf_predictions

# Preview the first few rows
print(forecast_result_2.head(10))
```

```
   YearMonth     country ModelFamily  Predicted Closing Subs Monthly
0     202312  California    OPPO A12                          90.09
1     202312       Texas    OPPO A12                          46.22
2     202312  California    OPPO A15                         114.09
3     202312  California    OPPO A15                         123.00
4     202312  California    OPPO A16                         279.42
5     202312  California    OPPO A16                         301.80
6     202312       Texas    OPPO A16                         243.67
7     202312       Texas    OPPO A16                         243.67
8     202312       Texas    OPPO A16                         237.61
9     202312       Texas    OPPO A16                         237.61
```

8. Hyperparameter Tuning

- Linear Regression: RidgeCV applied with alphas = np.logspace(-3, 3, 50)
- XGBoost & Random Forest:
    i. RandomizedSearchCV used with 30 iterations, 3-fold CV.
- Performance After Hyperparamter Tuning

Key Evaluation Metric			
	MAE	RMSE	R^2
Linear Regression	549.48	828.89	0.801
Random Forest	273.29	468.46	0.936
XGBoost	275.94	465.31	0.937

Reasons:

- Why Linear Regression performance decreases: Linear regression has high bias by nature thus, further penalising it reduces its learning capacity.
- Why XGBoost performance improves: hyper parameter fine-tuning allows better bias–variance balance, more robust generalization, and optimized complexity.
- Why Random Forest performance improves: The tuned hyperparameters helped strike a better bias–variance tradeoff, making the Random Forest more generalizable and sensitive to subtle patterns — especially useful in your time-series forecasting use case.

9. Feature Importance & Filtering

- Feature importances extracted per model.

In [122]:
```
feature_names = X.columns
```

In [123]:
```
# Extract model
lr_model = models['Linear Regression']

# Create a Series of absolute coefficient values
lr_importance_raw = pd.Series(lr_model.coef_, index=feature_names).abs()

# Normalize to sum to 1 → like percent contributions
lr_importance_pct = lr_importance_raw / lr_importance_raw.sum()

# Combine both into a DataFrame
lr_importance_df = pd.DataFrame({
    'Coefficient (Abs)': lr_importance_raw,
    'Importance (%)': lr_importance_pct
})

# Sort and display top 10
lr_importance_df = lr_importance_df.sort_values(by='Coefficient (Abs)', ascending=False)
print(lr_importance_df.head(10))
```

```
                                    Coefficient (Abs)  Importance (%)
ModelFamily_OPPO A15                      2347.115170        0.106016
ModelFamily_OPPO A12                      1234.745106        0.055772
ModelFamily_OPPO A5                       1085.333597        0.049023
ModelFamily_SAMSUNG GALAXY A10             992.431955        0.044827
Churn                                      924.268991        0.041748
ModelFamily_SAMSUNG GALAXY S21 ULTRA       915.033860        0.041331
Filed Claims                               898.934849        0.040604
ModelFamily_SAMSUNG GALAXY S22 ULTRA       874.744129        0.039511
ModelFamily_OPPO A16                       872.208085        0.039397
ModelFamily_SAMSUNG GALAXY S20 ULTRA       869.161940        0.039259
```

In [124]:
```
rf_model = models['Random Forest']

rf_importance = pd.Series(rf_model.feature_importances_, index=feature_names)
rf_importance.sort_values(ascending=False).head(10)
```

Out[124]:
```
Filed Claims                          0.538721
Churn                                 0.253465
Model Age (Days)                      0.052934
ModelFamily_OPPO A15                  0.039340
ModelFamily_APPLE IPHONE 12 PRO MAX   0.022629
country_Texas                         0.010502
Size                                  0.010447
Claims Swap                           0.009646
Claims Replacement                    0.008889
ModelFamily_OPPO A12                  0.008089
dtype: float64
```

In [125]:
```
xgb_model = models['XGBoost']

xgb_importance = pd.Series(xgb_model.feature_importances_, index=feature_names)
xgb_importance.sort_values(ascending=False).head(10)
```

Out[125]:
```
ModelFamily_OPPO A15                  0.457656
Filed Claims                          0.143957
ModelFamily_OPPO A12                  0.060884
ModelFamily_APPLE IPHONE 12 PRO MAX   0.054527
ModelFamily_SAMSUNG GALAXY A10        0.031311
Churn                                 0.030117
ModelFamily_OPPO A3S                  0.024374
ModelFamily_SAMSUNG GALAXY S20 ULTRA  0.023430
Colour_Blue                           0.021631
ModelFamily_OPPO A16                  0.020790
dtype: float32
```

- Features contributing <1% to all models were removed: 27 out 57 were removed
- Performance after feature filtering

Key Evaluation Metric			
	MAE	RMSE	R^2
Linear Regression	613.563	827.911	0.801
Random Forest	278.259	518.114	0.922
XGBoost	294.511	502.112	0.927

- Why the performance metrics performance decreases for all 3 models:
    - Small Features are still important collectively: Each feature <1% may look weak individually, but together they contribute additive predictive value — especially in ensemble models like RF and XGB
    - Non-linear Interactions were lost: Models like Random Forest & XGBoost rely on feature interactions. Removing low-importance variables may break key paths in the decision trees

- Over-filtering removed signal: 27 out of 57 (~47%) features were dropped, some of them may have been more useful than they appeared, especially under cross-feature dependencies

10. Final Performance Summary

- XGBoost is generally faster than Random Forest during fine-tuning. This is because XGBoost is highly optimized with a C++ backend, supports parallel processing, and allows early stopping — which helps skip unpromising parameter combinations. In contrast, Random Forest lacks early stopping and fully grows each tree for every combination, making it slower when exploring large parameter grids. So, despite being more complex, XGBoost fine-tunes more efficiently and completes faster in practice.
- However, XGBoost produced negative predicted values for "Closing Subs Monthly", despite the training data only containing positive values. This indicates that the model, while numerically strong, lacks constraint awareness and may not be aligned with realistic business outcomes. Random Forest, on the other hand, yielded only positive and plausible values, consistent with the nature of the target variable in the historical data. This makes Random Forest more reliable and suitable for business decision-making, particularly in scenarios where negative outputs are illogical or invalid.

In [113]:
```
# Make predictions using XG Boost
future_predictions = models['XGBoost'].predict(future_encoded)

# Output
forecast_result_1 = future_data[['YearMonth', 'country', 'ModelFamily']].copy()
forecast_result_1['Predicted Closing Subs Monthly'] = future_predictions

print(forecast_result_1.head(10))
```

	YearMonth	country	ModelFamily	Predicted Closing Subs Monthly
0	202312	California	OPPO A12	-43.704189
1	202312	Texas	OPPO A12	129.454697
2	202312	California	OPPO A15	68.050888
3	202312	California	OPPO A15	-71.584686
4	202312	California	OPPO A16	319.590729
5	202312	California	OPPO A16	342.318634
6	202312	Texas	OPPO A16	362.251984
7	202312	Texas	OPPO A16	362.251984
8	202312	Texas	OPPO A16	353.466492
9	202312	Texas	OPPO A16	353.466492

In [116]:
```
# Make predictions using Random Forest
rf_predictions = models['Random Forest'].predict(future_encoded)

# Output
forecast_result_2 = future_data[['YearMonth', 'country', 'ModelFamily']].copy()
forecast_result_2['Predicted Closing Subs Monthly'] = rf_predictions

# Preview the first few rows
print(forecast_result_2.head(10))
```

	YearMonth	country	ModelFamily	Predicted Closing Subs Monthly
0	202312	California	OPPO A12	90.09
1	202312	Texas	OPPO A12	46.22
2	202312	California	OPPO A15	114.09
3	202312	California	OPPO A15	123.00
4	202312	California	OPPO A16	279.42
5	202312	California	OPPO A16	301.80
6	202312	Texas	OPPO A16	243.67
7	202312	Texas	OPPO A16	243.67
8	202312	Texas	OPPO A16	237.61
9	202312	Texas	OPPO A16	237.61

Model	MAE	RMSE	R Square	Fine-Tuning	O-Complexity during fine-tuning	Realistic Output
Random Forest	273.29	468.46	0.936	Yes	O(n_estimator x n_samples x log n_sample)	Yes
XGBoost	275.94	465.31	0.937	Yes	O(n_estimator x max_depth x n_sample)	No

11. Conclusion

Based on all the criteria analysed, the best model to be chosen is Random Forest.

Justification:

1. Strong Predictive Performance
   - MAE: 273.29
   - RMSE: 468.46
   - R^2: 0.936

These values are nearly identical to those of XGBoost, with Random Forest achieving slightly lower MAE and only marginally higher RMSE. The difference in performance is minimal and within acceptable tolerance.

2. Realistic Output
   - Random Forest generated only positive predictions, which aligns with the business logic (demand cannot be negative).
   - XGBoost, despite its strong metrics, produced unrealistic negative values for predicted demand, which poses a risk in operational decision-making.

While both models are excellent, Random Forest strikes the best balance between accuracy, computational efficiency, and realistic applicability. It is the most suitable choice for deployment in this demand forecasting task